

รายงาน Lab 8 Table Relationships

รายวิชา: CP353002 Principles of Software Design

เรื่อง: ระบบจัดการสินค้าและความสัมพันธ์ตารางด้วย Spring Boot, JPA และ PostgreSQL

รหัสนักศึกษา: 673380048-6

Section: 1

ส่วนที่ 1 หลักการออกแบบ

1.1 ภาพรวมของระบบ

ระบบ Product Shop เป็นเว็บแอปพลิเคชันสำหรับเพิ่ม แสดง แก้ไข และลบสินค้า โดยจัดเก็บข้อมูลในฐานข้อมูล PostgreSQL ชื่อ lab8shop ประกอบด้วยตาราง products, product_details และ reviews

ระบบแบ่งการทำงานเป็นชั้น Controller, Service และ Repository โดยใช้ Entity เชื่อมโยงกับตารางฐานข้อมูล และใช้ Strategy Pattern แยกวิธีคำนวณส่วนลดแต่ละประเภท

1.2 การประยุกต์ใช้ SOLID Principles

Single Responsibility Principle — SRP

แยกข้อมูลและหน้าที่ของแต่ละคลาสอย่างชัดเจน โดย Product จัดเก็บข้อมูลหลักของสินค้า ProductDetail จัดเก็บรายละเอียดเพิ่มเติม และ Review จัดเก็บความคิดเห็นของผู้รีวิว

ส่วน Controller รับคำขอและเตรียมข้อมูลสำหรับหน้าเว็บ Service ตรวจสอบข้อมูลและดำเนินการตามกฎของระบบ และ Repository ติดต่อฐานข้อมูล

Open/Closed Principle — OCP

ใช้ DiscountStrategy เป็นข้อตกลงร่วมสำหรับการคำนวณส่วนลด สามารถเพิ่มคลาสส่วนลดใหม่และลงทะเบียนเป็น Spring Component โดยไม่ต้องเปลี่ยนวิธีเลือก Strategy ภายใน DiscountContext

อย่างไรก็ตาม หากเพิ่มประเภทส่วนลดที่ผู้ใช้เลือกได้ ต้องปรับรายการประเภทที่ Service ยอมรับและตัวเลือกในแบบฟอร์มให้รองรับด้วย

Liskov Substitution Principle — LSP

คลาส NoDiscountStrategy, MemberDiscountStrategy และ SeasonalSaleStrategy สามารถใช้งานผ่าน interface DiscountStrategy เดียวกันได้ โดยผู้เรียกใช้เมธอด calculateFinalPrice() โดยไม่จำเป็นต้องทราบคลาสจริง

Interface Segregation Principle — ISP

DiscountStrategy มีเฉพาะเมธอดที่เกี่ยวข้องกับส่วนลด ได้แก่ การคำนวณราคาสุทธิและการคืนซื้อส่วนลด ทำให้คลาสที่นำ interface ไปใช้ไม่ต้องรองรับงานที่ไม่เกี่ยวข้อง

Dependency Inversion Principle — DIP

ProductService ดึงข้อมูลผ่าน ProductRepository ซึ่งเป็น interface และ DiscountContext เรียกใช้ส่วนลดผ่าน DiscountStrategy จึงไม่ต้องผูกกับรายละเอียดการทำงานของ Repository หรือ Strategy แต่ละตัวโดยตรง

1.3 ความสัมพันธ์แบบ One-to-One และ One-to-Many

One-to-One ระหว่าง Product กับ ProductDetail

สินค้าแต่ละรายการเชื่อมโยงรายละเอียดสินค้าได้ไม่เกินหนึ่งรายการ เหมาะสำหรับแยกข้อมูลเพิ่มเติม เช่น รายละเอียดสินค้า การรับประกัน น้ำหนัก ขนาด และประเทศที่ผลิต ออกจากข้อมูลหลักของสินค้า

Foreign Key อยู่ในตาราง products ดังนี้

products.detail_id → product_details.id

ฝั่ง Product เป็นเจ้าของความสัมพันธ์ เนื่องจากประกาศ @JoinColumn ส่วน ProductDetail เป็นฝั่งอ้างอิงกลับ โดยใช้ mappedBy = "detail"

คอลัมน์ detail_id กำหนด unique = true เพื่อไม่ให้สินค้าหลายรายการอ้างอิงรายละเอียดรายการเดียวกัน

One-to-Many ระหว่าง Product กับ Review

สินค้าหนึ่งรายการมีรีวิวได้หลายรายการ แต่รีวิวแต่ละรายการเป็นของสินค้าเพียงหนึ่งรายการ เหมาะสำหรับข้อมูลที่สามารเพิ่มจำนวนได้ เช่น ความคิดเห็นของผู้ใช้

Foreign Key อยู่ในตาราง reviews ดังนี้

reviews.product_id → products.id

ฝั่ง Review เป็นเจ้าของความสัมพันธ์ เพราะจัดเก็บ Foreign Key ผ่าน @ManyToOne และ @JoinColumn ส่วน Product อ้างอิงรายการรีวิวผ่าน @OneToMany(mappedBy = "product")

1.4 Strategy Pattern สำหรับคำนวณส่วนลด

ระบบมีส่วนลดสามประเภท ได้แก่

ประเภท	คลาสที่ใช้	การคำนวณราคาสุทธิ
--------	------------	-------------------

NONE NoDiscountStrategy ราคาเดิม

MEMBER MemberDiscountStrategy ราคาเดิม $\times 0.90$

SEASONAL SeasonalSaleStrategy ราคาเดิม $\times 0.80$

DiscountContext รับ Strategy ทั้งหมดผ่าน Map<String, DiscountStrategy> โดยใช้ชื่อ Component เป็นคีย์สำหรับเลือกวิธีคำนวณ

ตัวอย่าง สินค้าราคา 1,000 บาท เมื่อเลือกส่วนลดสมาชิก 10% จะมีราคาสุทธิ 900 บาท และเมื่อแก้ไขราคา เป็น 1,200 บาท ราคาสุทธิจะเป็น 1,080 บาท

ราคาหลังส่วนลดเก็บไว้ในฟิลด์ discountedPrice ที่ประกาศ @Transient จึงเป็นค่าที่คำนวณเพื่อแสดงผล และไม่ได้จัดเก็บเป็นคอลัมน์ในฐานข้อมูล

1.5 Execution Flow ตั้งแต่ HTTP Request ถึงฐานข้อมูล

ตัวอย่างการเพิ่มสินค้า มีลำดับการทำงานดังนี้

1. ผู้ใช้เปิด /products/add และกรอกข้อมูลสินค้า รายละเอียดสินค้า และรีวิวแรก
2. เมื่อกดบันทึก แบบฟอร์มส่ง HTTP POST ไปที่ /products/save
3. ProductController รับข้อมูลผ่าน @ModelAttribute แล้วเรียก ProductService.createProduct()
4. Service ตรวจสอบข้อมูล เช่น ชื่อสินค้า ราคา จำนวนในคลัง และคะแนนรีวิว จากนั้นเชื่อม Product กับ ProductDetail และ Review
5. Service เรียก ProductRepository.save() โดย JPA จัดการบันทึก Entity ที่สัมพันธ์กันตามการตั้งค่า Cascade ภายใน Transaction
6. Hibernate ส่งคำสั่ง SQL ไปยัง PostgreSQL และบันทึกข้อมูลลงในตารางที่เกี่ยวข้อง
7. Controller เปลี่ยนเส้นทางกลับไป /products พร้อมข้อความแจ้งผล
8. ระบบอ่านรายการสินค้า คำนวณราคาหลังส่วนลด และส่งข้อมูลให้ Thymeleaf แสดงผล

ส่วนที่ 2 Code และคำอธิบาย

โค้ดต่อไปนี้จะแสดงส่วนสำคัญจากโปรเจกต์ โดยละ package, import และเมธอดทั่วไปบางส่วนเพื่อให้เห็น โครงสร้างและความสัมพันธ์ชัดเจน

2.1 Product Entity

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String category;
    private String brand;
    private Integer stock;
    private Double price;
    private String discountType = "NONE";

    @Transient
    private Double discountedPrice;

    @OneToOne(
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
    @JoinColumn(name = "detail_id", unique = true)
    private ProductDetail detail;

    @OneToMany(
        mappedBy = "product",
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
```

```

    )

    private List<Review> reviews = new ArrayList<>();

    public void setDetail(ProductDetail detail) {
        if (this.detail != null) {
            this.detail.setProduct(null);
        }

        this.detail = detail;

        if (detail != null) {
            detail.setProduct(this);
        }
    }

    public void addReview(Review review) {
        reviews.add(review);
        review.setProduct(this);
    }

    // ละ constructor และ getter/setter อื่น
}

```

คำอธิบาย

- @Entity กำหนดให้คลาสเป็น Entity ที่ JPA จัดการ
- @Table(name = "products") ระบุชื่อตาราง
- @Id และ @GeneratedValue กำหนด Primary Key และให้ฐานข้อมูลสร้างหมายเลขอัตโนมัติ
- @Column(nullable = false) กำหนดให้ชื่อสินค้าไม่เป็น NULL
- @OneToOne เชื่อมสินค้าเข้ากับรายละเอียดสินค้า
- @JoinColumn กำหนดคอลัมน์ Foreign Key ชื่อ detail_id

- @OneToMany เชื่อมสินค้ากับรีวิวหลายรายการ โดย mappedBy อ้างถึงฟิลด์ product ในคลาส Review
- CascadeType.ALL ส่งต่อการดำเนินการของ JPA เช่น บันทึกลงและลบ ไปยัง Entity ที่สัมพันธ์กัน
- orphanRemoval = true ให้ JPA ลบข้อมูลลูกที่ถูกนำออกจากความสัมพันธ์
- @Transient ทำให้ discountedPrice ไม่ถูกจัดเก็บในฐานข้อมูล

เมธอด setDetail() และ addReview() ช่วยกำหนดความสัมพันธ์ทั้งสองฝั่งให้สอดคล้องกัน โดยเฉพาะการตั้ง Review.product ซึ่งเป็นฝั่งที่เก็บ Foreign Key

2.2 ProductDetail Entity

```
@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(columnDefinition = "text")
    private String description;

    private String warranty;
    private Double weight;
    private String dimensions;
    private String manufacturedCountry;

    @OneToOne(mappedBy = "detail")
    private Product product;

    // ละ constructor และ getter/setter
}
```

คำอธิบาย

ProductDetail จัดเก็บรายละเอียดสินค้า ได้แก่ คำอธิบาย การรับประกัน น้ำหนัก ขนาด และประเทศที่ผลิต

@Column(columnDefinition = "text") กำหนดให้คำอธิบายสินค้าเป็นชนิดข้อความ ส่วน

@OneToOne(mappedBy = "detail") ระบุว่าความสัมพันธ์นี้ถูกควบคุมโดยฟิลด์ detail ของ Product จึงไม่สร้าง Foreign Key ขึ้นในตาราง product_details

ฟิลด์พื้นฐานที่ไม่ได้เขียน @Column ยังถูก JPA จับคู่เป็นคอลัมน์ตามค่าเริ่มต้นได้

2.3 Review Entity

```
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String reviewer;

    @Column(nullable = false)
    private Integer rating;

    @Column(columnDefinition = "text")
    private String comment;

    private LocalDate reviewDate;

    @ManyToOne(
        fetch = FetchType.LAZY,
        optional = false
    )
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;
```

```

@PrePersist
private void initializeReviewDate() {
    if (reviewDate == null) {
        reviewDate = LocalDate.now();
    }
}

// ละ constructor และ getter/setter
}

```

คำอธิบาย

Review จัดเก็บชื่อผู้รีวิว คะแนน ความเห็น และวันที่รีวิว โดย @ManyToOne กำหนดให้รีวิวหลายรายการอ้างอิงสินค้าเดียวกันได้

@JoinColumn(name = "product_id", nullable = false) กำหนดให้ทุกรีวิวต้องอ้างอิงสินค้า ส่วน FetchType.LAZY ระบุให้โหลดข้อมูลสินค้าที่สัมพันธ์กันเมื่อจำเป็น

@PrePersist กำหนดวันที่ปัจจุบันก่อนบันทึกรีวิวใหม่ หากยังไม่มีวันที่ และ Service ตรวจสอบให้คะแนนอยู่ระหว่าง 1 ถึง 5

ฝั่ง Review.product ไม่กำหนด Cascade สำหรับการลบ เพื่อไม่ให้เกิดการลบรีวิวส่งผลลบสินค้าตามไปด้วย

2.4 Repository

```

public interface ProductRepository
    extends JpaRepository<Product, Long> {
}

public interface ProductDetailRepository
    extends JpaRepository<ProductDetail, Long> {
}

public interface ReviewRepository
    extends JpaRepository<Review, Long> {
}

```


แต่ละ interface อยู่คนละไฟล์ และรับความสามารถพื้นฐานจาก JpaRepository เช่น findAll(), findById(), save() และ delete()

ในการบันทึกสินค้าพร้อมรายละเอียดและรีวิว Service ใช้ ProductRepository เป็นจุดเริ่มต้น และให้ JPA จัดการข้อมูลที่สัมพันธ์กันผ่าน Cascade

2.5 ProductService

```
@Service
@Transactional(readOnly = true)
public class ProductService {

    private final ProductRepository productRepository;
    private final DiscountContext discountContext;

    public ProductService(
        ProductRepository productRepository,
        DiscountContext discountContext) {
        this.productRepository = productRepository;
        this.discountContext = discountContext;
    }

    public List<Product> getAllProducts() {
        List<Product> products = productRepository.findAll();
        products.forEach(this::prepareForDisplay);
        return products;
    }

    @Transactional
    public Product updateProduct(Long id, Product form) {
        Product product = findProduct(id);
        validateProduct(form);
        copyProductFields(form, product);
        copyDetail(form.getDetail(), product);
    }
}
```

```

        return productRepository.save(product);
    }

    @Transactional
    public void deleteProduct(Long id) {
        productRepository.delete(findProduct(id));
    }

    // ละ createProduct และเมธอดช่วยอื่น
}

```

คำอธิบายการทำงาน

@Service กำหนดให้คลาสเป็นส่วนจัดการกฎและการทำงานของระบบ ส่วน @Transactional(readOnly = true) กำหนดค่าเริ่มต้นสำหรับการอ่านข้อมูล และเมธอดที่เพิ่ม แก้ไข หรือลบข้อมูลใช้ @Transactional เพื่อเปิดการเขียนข้อมูล

หน้าที่สำคัญของ Service ได้แก่

- getAllProducts() อ่านสินค้าและเตรียมราคาหลังส่วนลดสำหรับแสดงผล
- createProduct() ตรวจสอบข้อมูล สร้างสินค้า เชื่อมรายละเอียดและรีวิวก แล้วบันทึกผ่าน Repository
- updateProduct() แก้ไขสินค้าและรายละเอียดเดิม โดยคงรีวิวที่มีอยู่
- deleteProduct() ลบสินค้า โดยความสัมพันธ์ที่กำหนด Cascade จะถูกจัดการผ่าน JPA

ระหว่างสร้างสินค้า หากไม่ได้กรอกชื่อผู้รีวิวและความเห็น ระบบจะข้ามรีวิวว่าง หากมีข้อมูลรีวิว ระบบจะตรวจสอบชื่อผู้รีวิวและคะแนนก่อนบันทึก

การกำหนด Cascade ใน Entity เป็นการทำงานของ JPA จึงไม่ควรสับสนกับการกำหนด ON DELETE CASCADE ที่ Foreign Key ของฐานข้อมูลโดยตรง

2.6 ProductController

```

@Controller
@RequestMapping("/products")
public class ProductController {

```

```
private final ProductService productService;

public ProductController(ProductService productService) {
    this.productService = productService;
}

@GetMapping({ "", "/" })
public String listProducts(Model model) {
    model.addAttribute(
        "products",
        productService.getAllProducts()
    );
    return "products/list";
}

@PostMapping("/save")
public String saveProduct(
    @ModelAttribute("product") Product product,
    RedirectAttributes redirect) {

    productService.createProduct(product);

    redirect.addFlashAttribute(
        "message",
        "เพิ่มสินค้าสำเร็จ"
    );

    return "redirect:/products";
}
```

```
// ละเมียดแสดงฟอร์ม แก้ไข ลบ และจัดการซื้อผิดพลาด
```

```
}
```

คำอธิบาย

@Controller กำหนดให้คลาสรับคำขอของหน้าเว็บ และ @RequestMapping("/products") กำหนดเส้นทางหลัก

@GetMapping ใช้รับคำขอแสดงข้อมูลหรือแบบฟอร์ม ส่วน @PostMapping ใช้รับคำขอที่เปลี่ยนแปลงข้อมูล

@ModelAttribute ผูกข้อมูลจากแบบฟอร์มเข้ากับออบเจกต์ และ Model ใช้ส่งข้อมูลไปยัง Thymeleaf หลังบันทึกสำเร็จ Controller เปลี่ยนเส้นทางกลับหน้ารายการ พร้อมส่งข้อความผ่าน RedirectAttributes Controller ยังใช้ @InitBinder จำกัดฟิลด์ที่รับจากแบบฟอร์ม เพื่อให้ Service เป็นผู้จัดการรหัสและความสัมพันธ์ของข้อมูล

เส้นทางที่ระบบรองรับ

Method	URL	หน้าที่
GET	/products	แสดงรายการสินค้า
GET	/products/add	แสดงแบบฟอร์มเพิ่มสินค้า
POST	/products/save	บันทึกสินค้า รายละเอียด และรีวิวแรก
GET	/products/edit/{id}	แสดงแบบฟอร์มแก้ไข
POST	/products/update/{id}	บันทึกการแก้ไข
GET	/products/delete/{id}	แสดงหน้ายืนยันการลบ
POST	/products/delete/{id}	ลบสินค้า

2.7 Constructor Injection

ProductController รับ ProductService ผ่าน Constructor ส่วน ProductService รับ ProductRepository และ DiscountContext ผ่าน Constructor เช่นเดียวกัน

Spring เป็นผู้สร้างและส่งออบเจกต์ที่ต้องใช้เข้ามา ทำให้แต่ละคลาสไม่ต้องสร้าง Dependency ด้วย new ภายในคลาสเอง และเห็นได้ชัดว่าคลาสต้องพึ่งพาอะไรบ้าง

การประกาศฟิลด์เป็น final ช่วยให้ Dependency ถูกกำหนดตั้งแต่สร้างออบเจกต์ และสามารถส่งออบเจกต์
จำลองเข้าไปแทนในการทดสอบได้

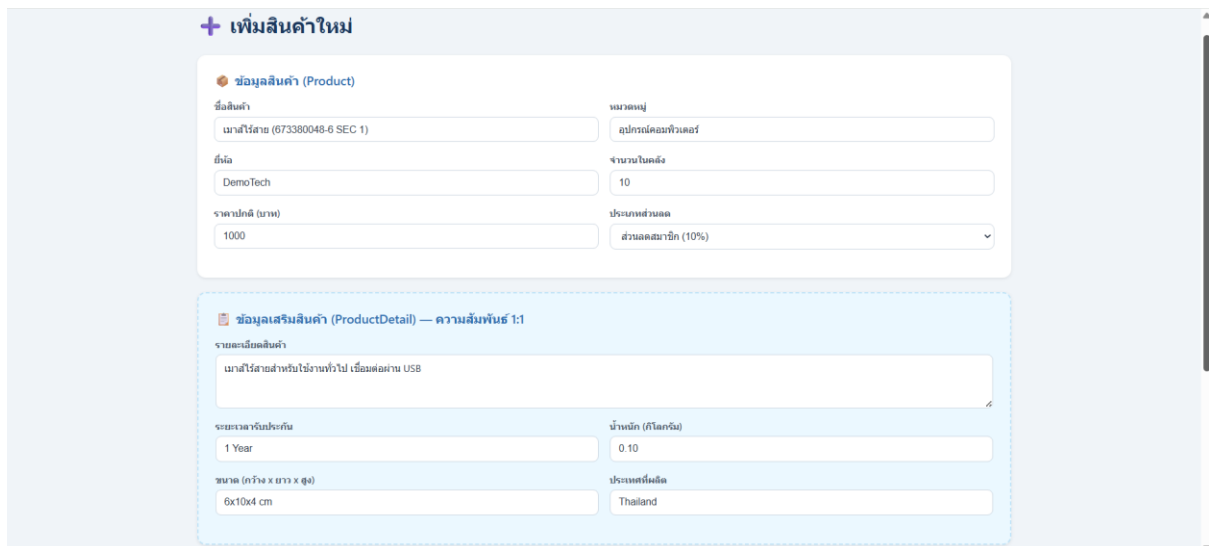
ส่วนที่ 3 ภาพหน้าจอและผลการทำงาน

3.1 Create การเพิ่มสินค้าและรายละเอียดสินค้า

ทดสอบเพิ่มสินค้าโดยใช้ชื่อ เมสรีไร้สาย (673380048-6 SEC 1) เพื่อแสดงรหัสนักศึกษาและ Section ตาม
ข้อกำหนด

ข้อมูลสินค้าเป็นหมวดอุปกรณ์คอมพิวเตอร์ ยี่ห้อ DemoTech จำนวน 10 ชิ้น ราคา 1,000 บาท และเลือก
ส่วนลดสมาชิก 10%

รายละเอียดเพิ่มเติมประกอบด้วยคำอธิบายสินค้า การรับประกัน 1 Year น้ำหนัก 0.10 กิโลกรัม ขนาด
6x10x4 cm และประเทศที่ผลิต Thailand



ภาพที่ 1 แบบฟอร์มเพิ่มสินค้าและรายละเอียดสินค้า แสดงข้อมูล Product และ ProductDetail ก่อนบันทึก

จากนั้นกรอกรีวิวแรก โดยให้คะแนน 5 ดาว และแสดงความคิดเห็นว่า “ใช้งานสะดวก จับถนัดมือ”

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

เนาส์ให้สายสำหรับโรงงานทั่วไป เชื่อมต่อผ่าน USB

ระยะเวลาประกัน: 1 Year

น้ำหนัก (กิโลกรัม): 0.10

ขนาด (กว้าง x ยาว x สูง): 6x10x4 cm

ประเทศที่ผลิต: Thailand

★ รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N

เขียนรีวิวเกี่ยวกับสินค้า (สามารถเขียนรีวิวเพิ่มเติมภายหลังได้)

ชื่อผู้รีวิว: ปวิทย์

คะแนน (1-5): ★★★★★ (5)

ความเห็น: โรงงานสะดวก รันยอดเยี่ยม

ปุ่ม: บันทึกสินค้า, ยกเลิก

ภาพที่ 2 การกรอกรีวิวแรกพร้อมสินค้า เพื่อเชื่อมข้อมูล Review เข้ากับ Product

3.2 Read การแสดงรายการสินค้า

หลังบันทึก ระบบแสดงสินค้าที่เพิ่มในหน้ารายการ พร้อมจำนวนในคลัง ราคาปกติ ประเภทส่วนลด ราคาสุทธิ ข้อมูลการรับประกัน น้ำหนัก และจำนวนรีวิว

สินค้าราคา 1,000 บาท เมื่อใช้ส่วนลดสมาชิก 10% มีราคาสุทธิ 900 บาท และแสดงจำนวนรีวิว 1 รายการ

Product Shop Lab 8 — 1:1 & 1:N Relationship

เพิ่มสินค้าสำเร็จ

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
2	เนาส์ให้สาย (673380048-6 SEC 1)	อุปกรณ์คอมพิวเตอร์	DemoTech	10	1000.00	MEMBER	900.00	1 Year	0.1	1 รีวิว	แก้ไข / รีวิว ลบ

ภาพที่ 3 รายการสินค้าหลังเพิ่มสำเร็จ แสดงราคาสุทธิ 900 บาทและรีวิว 1 รายการ

3.3 Update การแก้ไขสินค้า

ทดสอบแก้ไขจำนวนสินค้าในคลังจาก 10 เป็น 15 ขึ้น ราคาปกติจาก 1,000 เป็น 1,200 บาท และการรับประกันจาก 1 Year เป็น 2 Year โดยยังใช้ส่วนลดสมาชิก 10%

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า:

หมวดหมู่:

ยี่ห้อ:

จำนวนในสต็อก:

ราคาปกติ (บาท):

ส่วนลดสมาชิก (10%):

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า:

ระยะเวลาประกัน:

น้ำหนัก (กิโลกรัม):

ขนาด:

ประเทศที่ผลิต:

ภาพที่ 4 แบบฟอร์มแก้ไขข้อมูลสินค้าและรายละเอียดการรับประกัน

หลังบันทึก ระบบแสดงจำนวนสินค้า 15 ชิ้น ราคาปกติ 1,200 บาท ราคาสุทธิ 1,080 บาท และการรับประกัน 2 Year โดยจำนวนรีวิวยังคงเป็น 1 รายการ

Product Shop Lab 8 — 1:1 & 1:N Relationship

แก้ไขสินค้าสำเร็จ

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
2	นาฬิกาใส่สาย (673380048-6 SEC 1)	อุปกรณ์คอมพิวเตอร์	DemoTech	15	1200.00	MEMBER	1080.00	2 Year	0.1	1 รีวิว	แก้ไข / รีวิว ลบ

ภาพที่ 5 ผลหลังแก้ไขสินค้า แสดงข้อมูลใหม่และคำนวณราคาหลังส่วนลดใหม่ได้ถูกต้อง

3.4 Delete การลบสินค้า

เมื่อเลือกการลบ ระบบแสดงข้อมูลสินค้าให้ตรวจสอบก่อนยืนยัน เพื่อให้ผู้ใช้ทราบว่ากำลังจะลบสินค้ารายการใด

ยืนยันการชำระเงิน

คุณต้องการชำระเงินด้วยวิธีไหน? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า:	เมาส์ไร้สาย (673380048-6 SEC 1)
หมวดหมู่:	อุปกรณ์คอมพิวเตอร์
ยี่ห้อ:	DemoTech
ราคา:	1200.00 บาท
รายละเอียด:	เมาส์ไร้สายสำหรับใช้งานทั่วไป เชื่อมต่อผ่าน USB
รับประกัน:	2 Year

รายละเอียดสินค้าและวิธีใช้งานของสินค้านี้จะถูกส่งพร้อมกับ

ยืนยัน **ยกเลิก**

ภาพที่ 6 หน้ายืนยันการชำระเงินเมาส์ไร้สายก่อนดำเนินการ

หลังยืนยัน ระบบแสดงข้อความ “ชำระเงินสำเร็จ” และไม่พบสินค้ารายการดังกล่าวในหน้ารายการ

Product Shop Lab 8 — 1:1 & 1:N Relationship

ชำระเงินสำเร็จ

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (ปี)	น้ำหนัก (กก)	รีวิว (อัน)	จัดการ
ยังไม่มีสินค้า — กด "+ เพิ่มสินค้าใหม่" เพื่อเริ่มต้น											

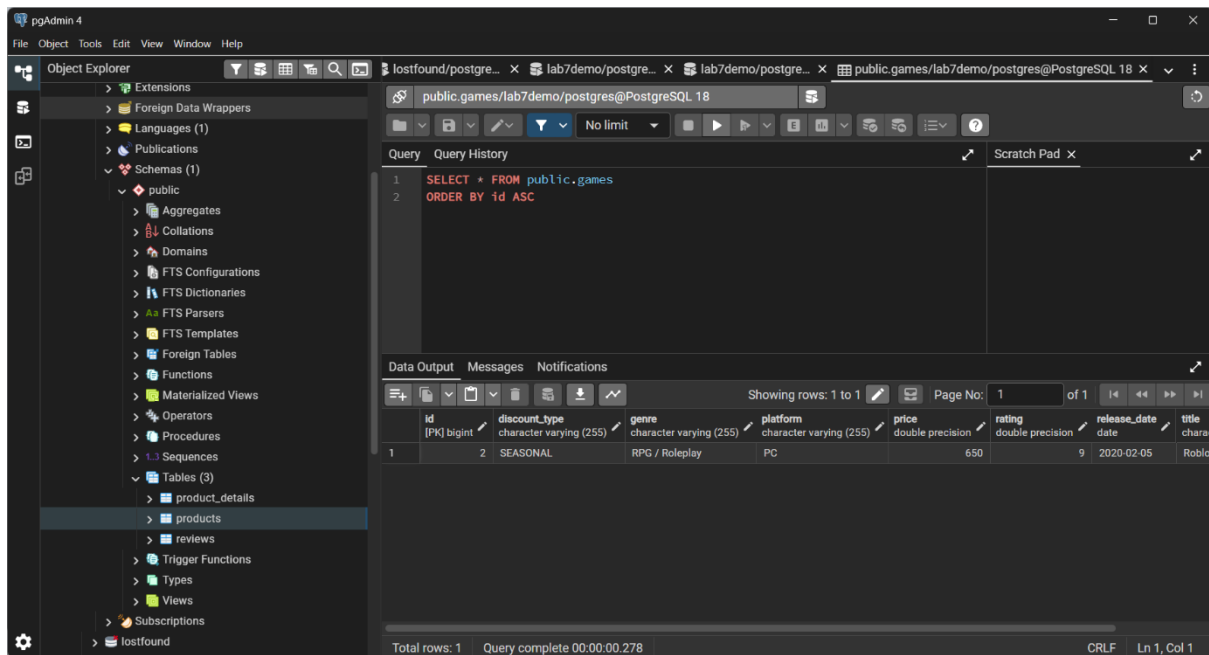
ภาพที่ 7 หน้ารายการหลังชำระเงิน แสดงข้อความสำเร็จและไม่มีสินค้ารายการเดิม

3.5 Database โครงสร้างตารางและ Foreign Key

ภาพฐานข้อมูลต่อไปนี้บันทึกไว้ก่อนลบสินค้า โดยใช้ Object Explorer ใน pgAdmin ตรวจสอบตาราง และใช้หน้าต่างรายละเอียด Constraint ตรวจสอบ Foreign Key

โครงสร้างสามตาราง

Object Explorer ด้านซ้ายแสดงตาราง product_details, products และ reviews ซึ่งสอดคล้องกับ Entity ทั้งหมดของระบบ

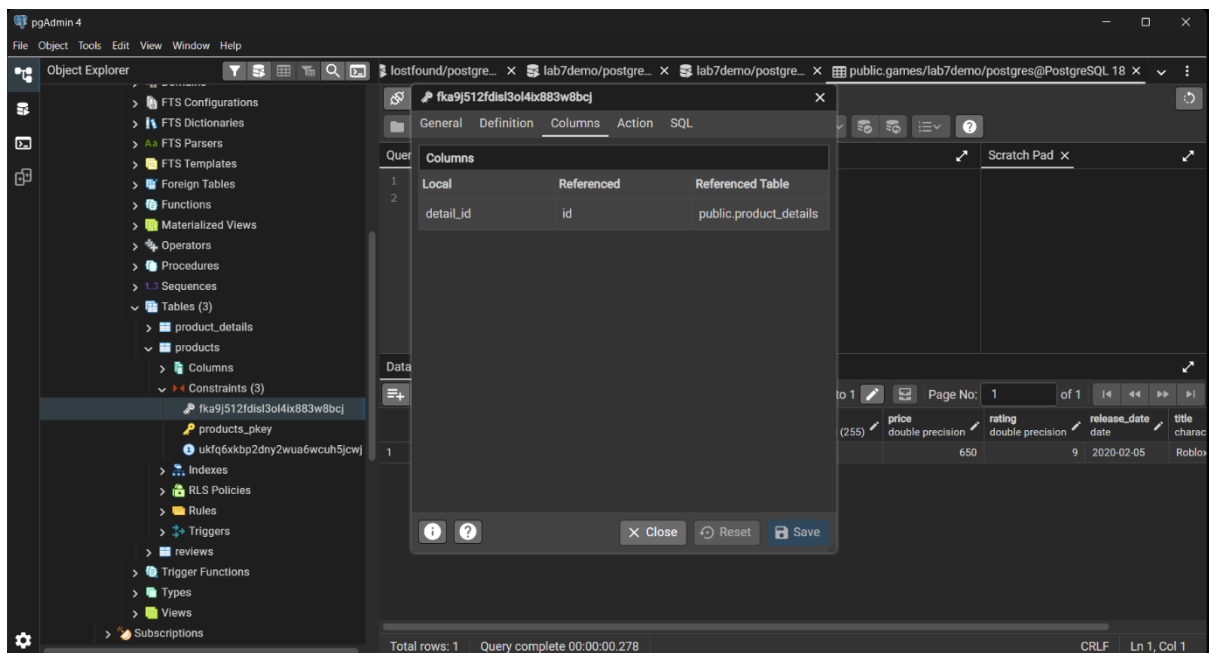


ภาพที่ 8 รายชื่อตารางทั้งสามใน Object Explorer ด้านซ้ายของ pgAdmin

ภาพนี้ใช้อ้างอิงรายชื่อตารางด้านซ้าย ส่วนแท็บข้อมูลด้านขวาเป็น public.games ของ Lab 7 จึงไม่ได้ใช้เป็นหลักฐานข้อมูลสินค้าใน Lab 8

Foreign Key ระหว่าง Product กับ ProductDetail

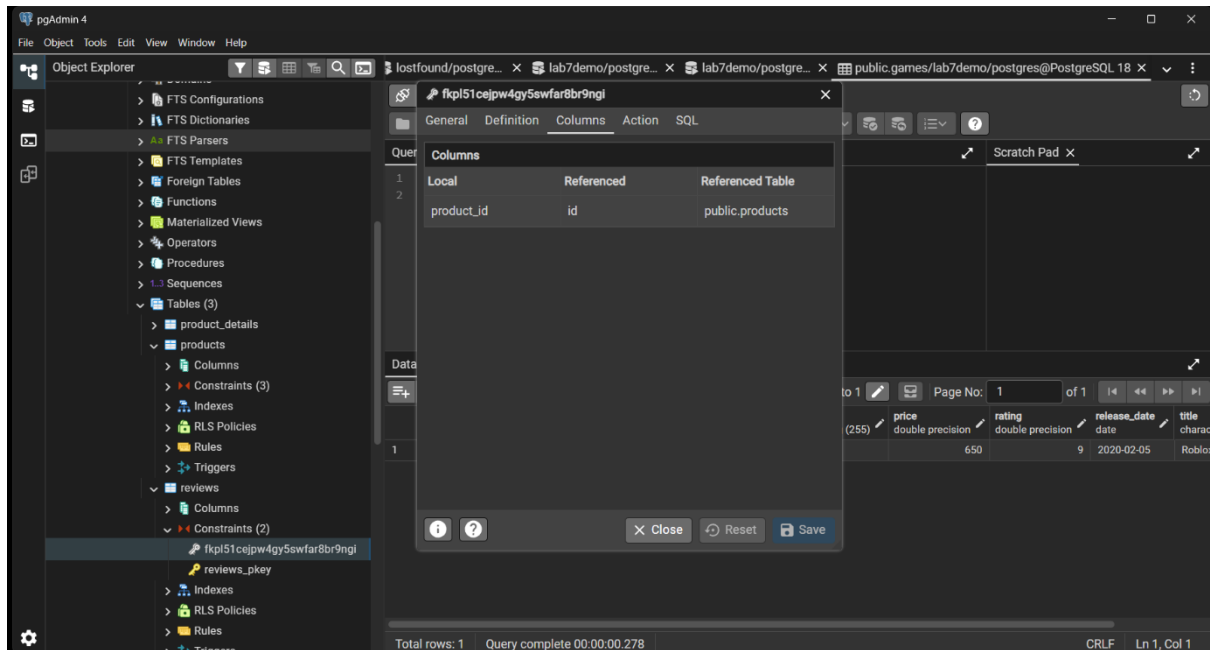
หน้าต่างรายละเอียด Foreign Key ของตาราง products แสดงว่า Local Column คือ detail_id และอ้างอิงคอลัมน์ id ของตาราง public.product_details



ภาพที่ 9 Foreign Key products.detail_id → product_details.id สำหรับความสัมพันธ์ One-to-One

Foreign Key ระหว่าง Review กับ Product

หน้าต่างรายละเอียด Foreign Key ของตาราง reviews แสดงว่า Local Column คือ product_id และอ้างอิงคอลัมน์ id ของตาราง public.products



ภาพที่ 10 Foreign Key reviews.product_id → products.id สำหรับความสัมพันธ์ One-to-Many โดย Foreign Key อยู่ที่ฝั่ง Review